

From Sensor Data to a Dataset

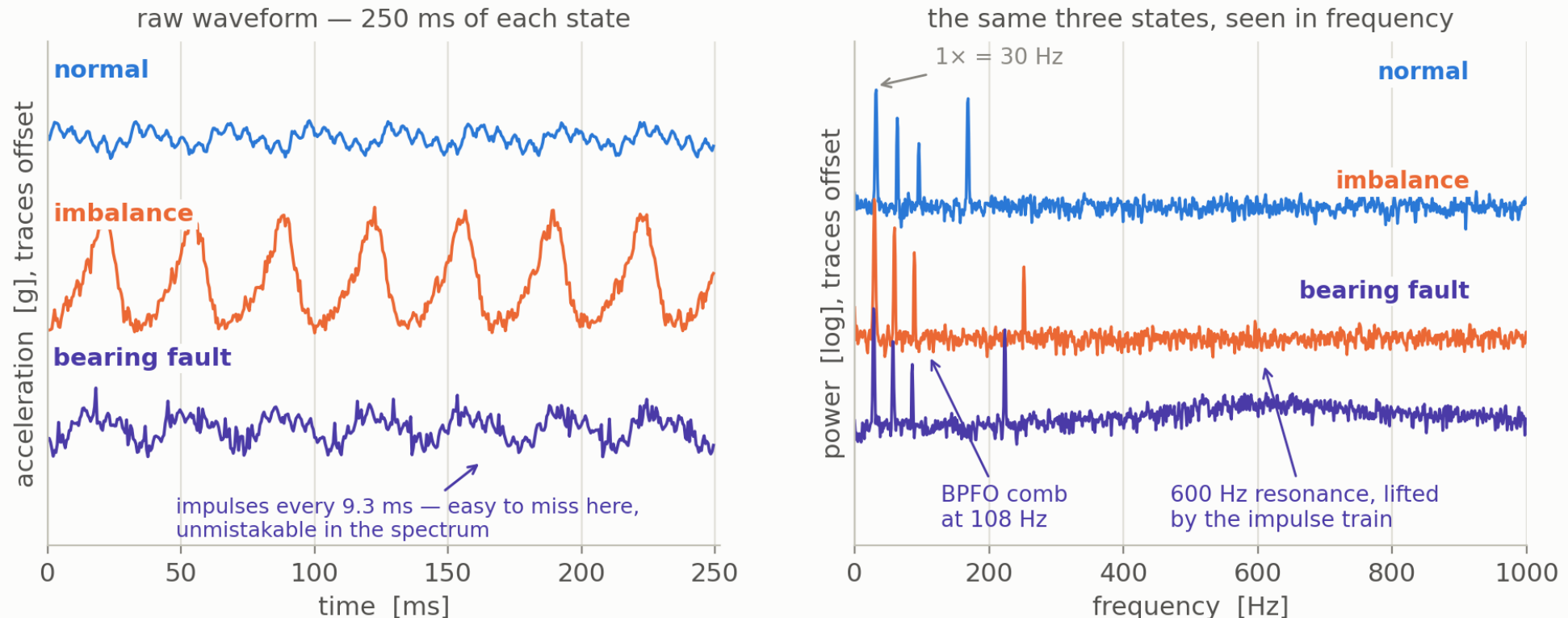
Turning an accelerometer stream into a table a classifier will accept

- Choosing the sampling rate from the fault, not the shaft
- Windowing, labelling, and the learning table
- Seven time-domain and five frequency-domain features
- Splitting by run — and the accuracy you invent when you do not
- When feature scaling matters, and when it is a ritual

Notebook: `Week8_From_Sensor_Data_to_a_Dataset.ipynb`

One machine, three states, no obvious threshold

- The three states differ, but no single sample tells you which one you are in — the information is in the shape of a whole window of samples.

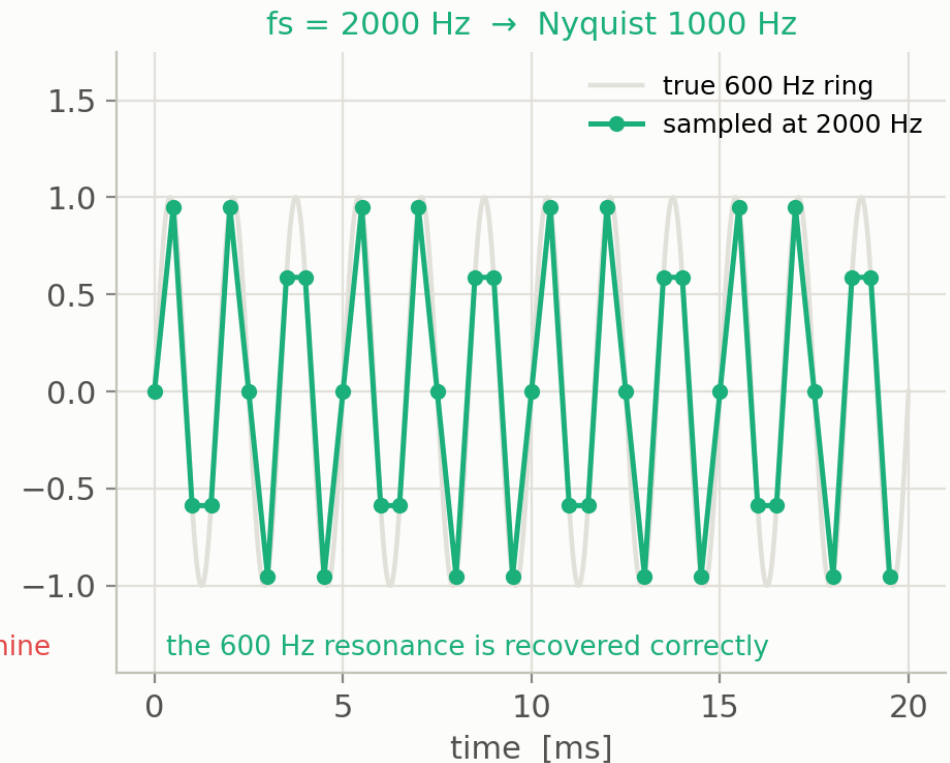
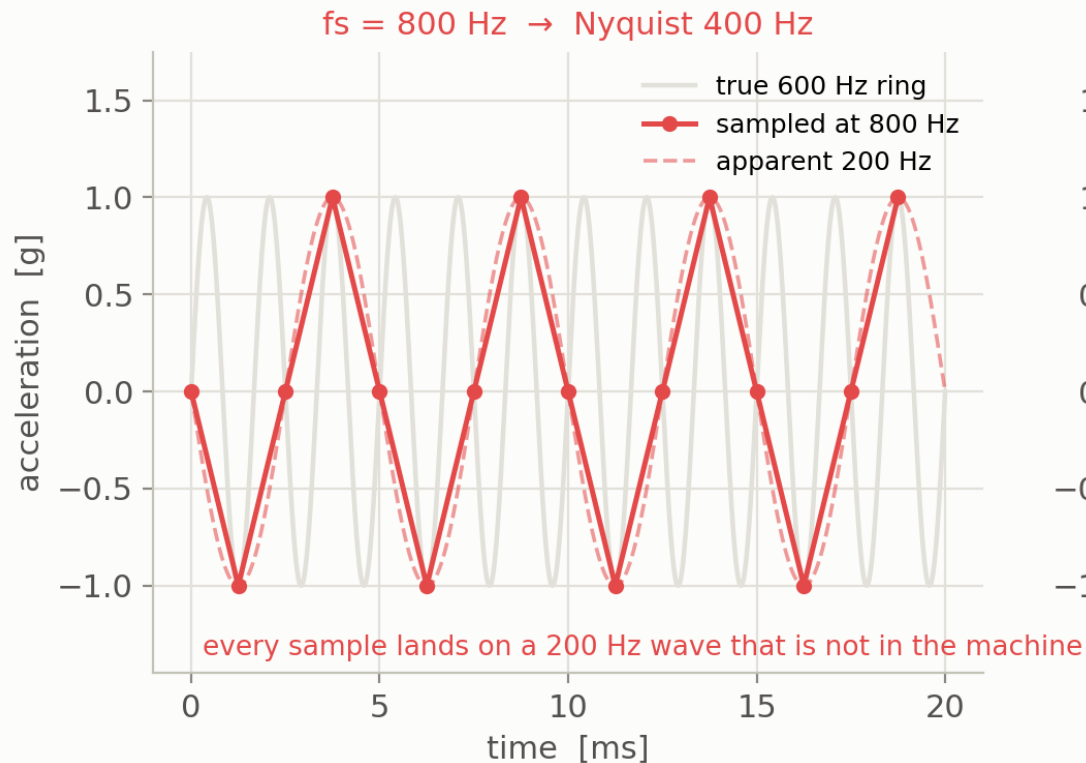


No single sample tells you which state you are in. The information lives in the shape of a whole window.

Imbalance loads the 1x shaft line; the bearing spall puts its energy into a 600 Hz housing resonance struck 108 times a second. Two faults, two different places to look.

Choose the sampling rate from the fault

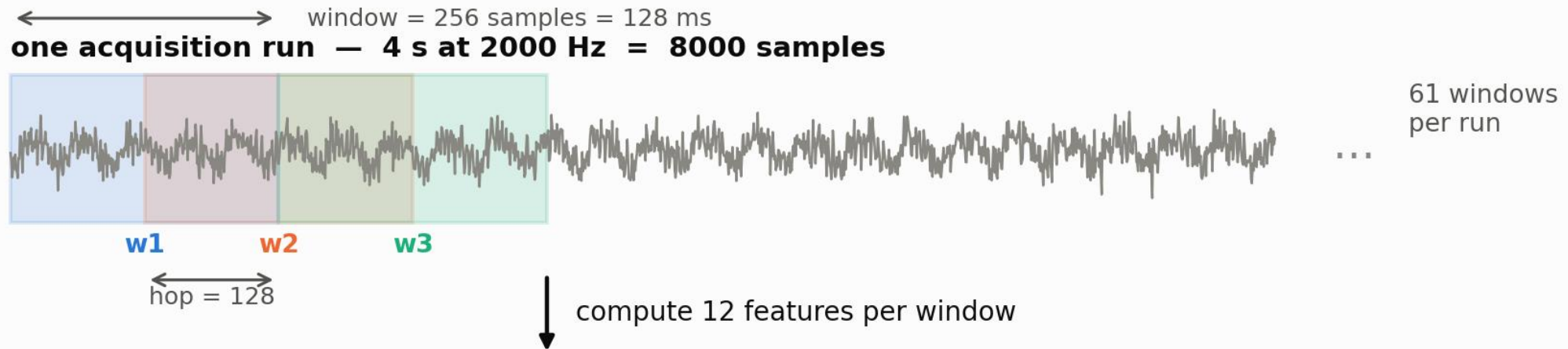
- Anything above $f_s/2$ does not vanish — it folds back and appears at a frequency that is not in the machine.



Choose f_s from the highest frequency that carries fault information (here the 600 Hz housing resonance), not from the shaft speed.

Sample for the highest frequency that carries diagnostic information — here the 600 Hz resonance, so $f_s = 2000 \text{ Hz}$ — and put a real anti-alias filter in front of the ADC.

One window in, one row out



the learning table — one row per window

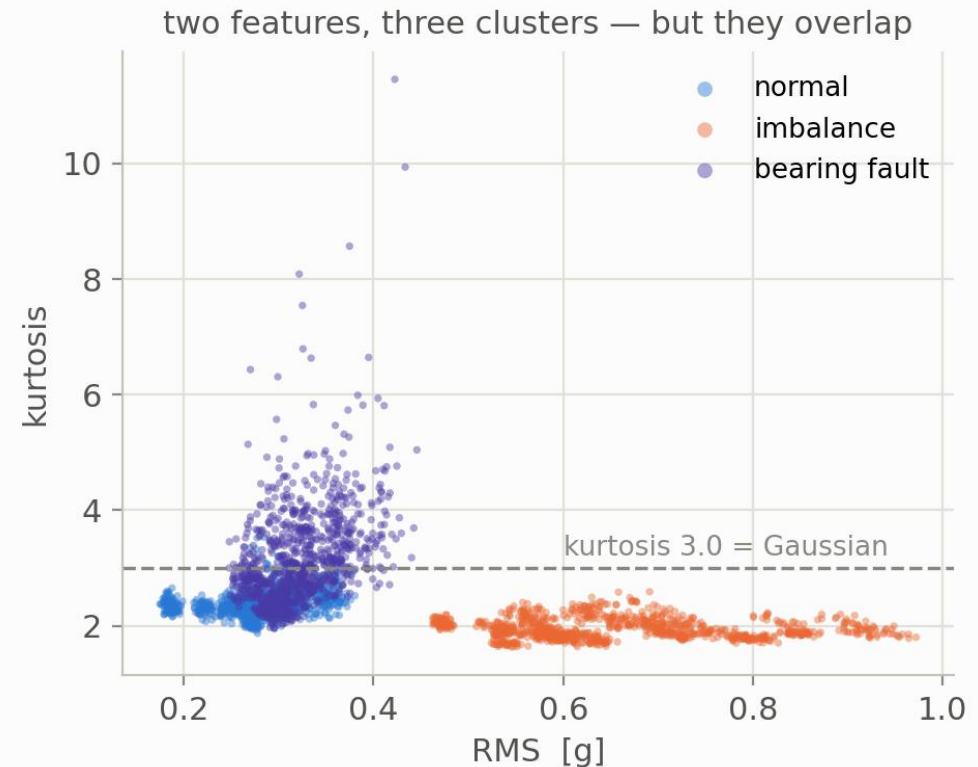
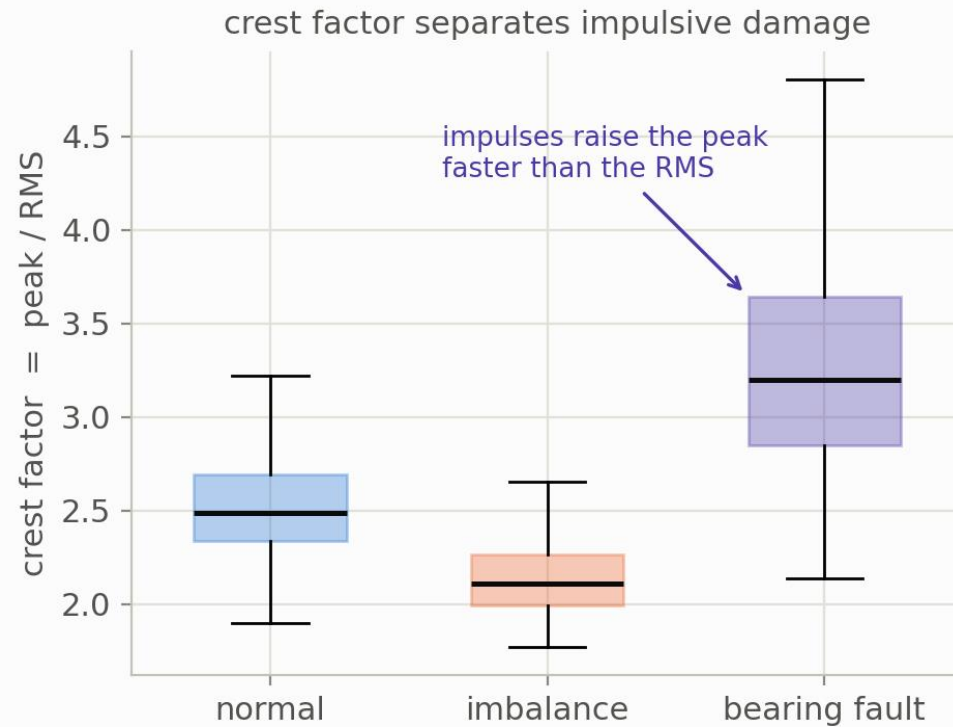
run	win	rms	crest	kurt	zcr	e_1x	e_bpfo	e_hi	label
0	w1	0.27	3.00	2.86	0.33	0.38	0.02	0.25	normal
0	w2	0.27	2.51	2.67	0.30	0.38	0.01	0.23	normal
0	w3	0.28	2.50	2.54	0.29	0.32	0.01	0.19	normal

Neighbouring rows share half their samples. Remember that when you split the data — it is the trap on the next slide.

256 samples, 50 % overlap, 61 rows per 4-second run, 2928 rows in total from 48 runs — and neighbouring rows share half their samples.

Time domain: how much, and what shape

- mean · rms · std · ptp · crest · kurtosis · zero-crossing rate — all computed after removing the DC component.

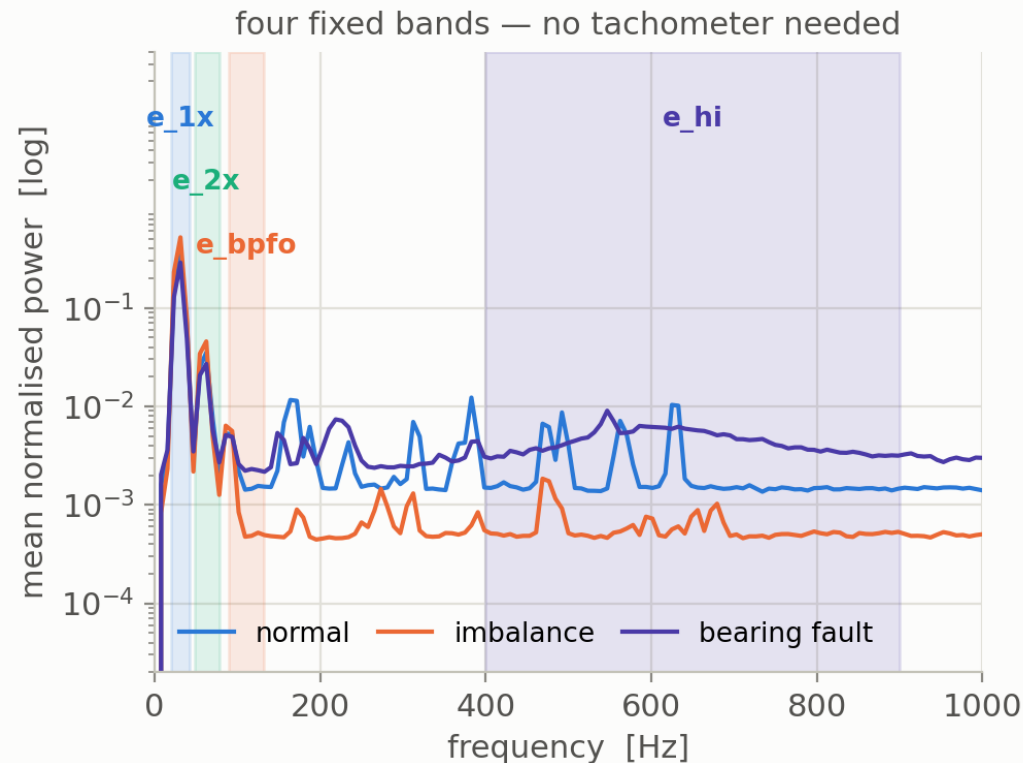


RMS answers "how much"; crest factor and kurtosis answer "what shape". A fault diagnosis needs both.

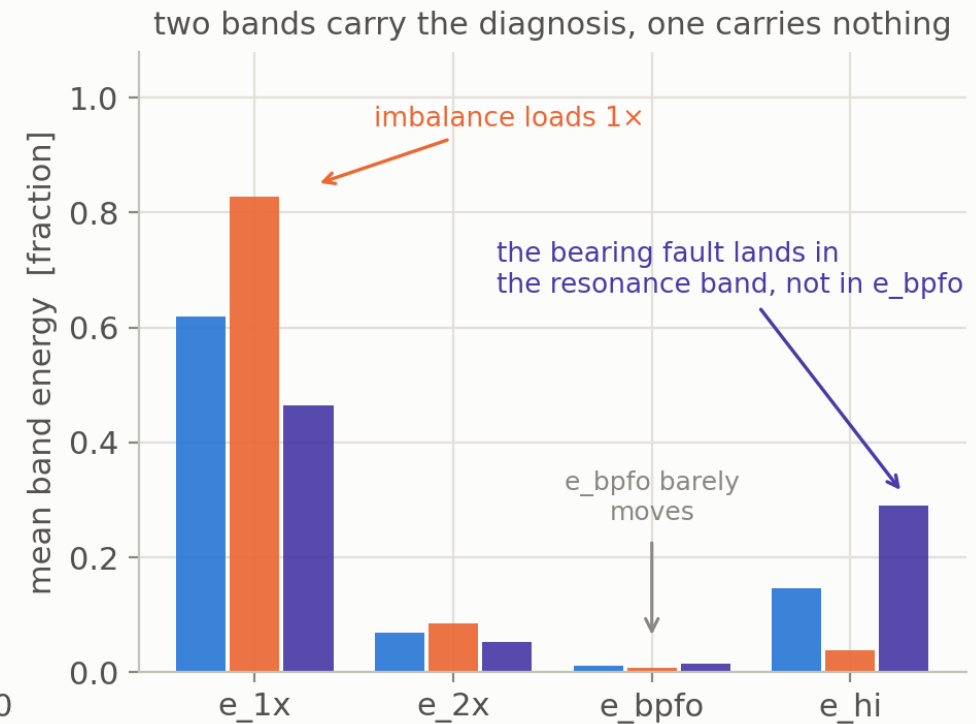
RMS rises for imbalance (0.277 → 0.658 g) but barely moves for the bearing fault; crest factor does the opposite (2.537 → 3.294).

Frequency domain: four bands and a peak

- Fixed bands in hertz, not shaft orders — our rig has no tachometer and the shaft wanders between 28 and 32 Hz.



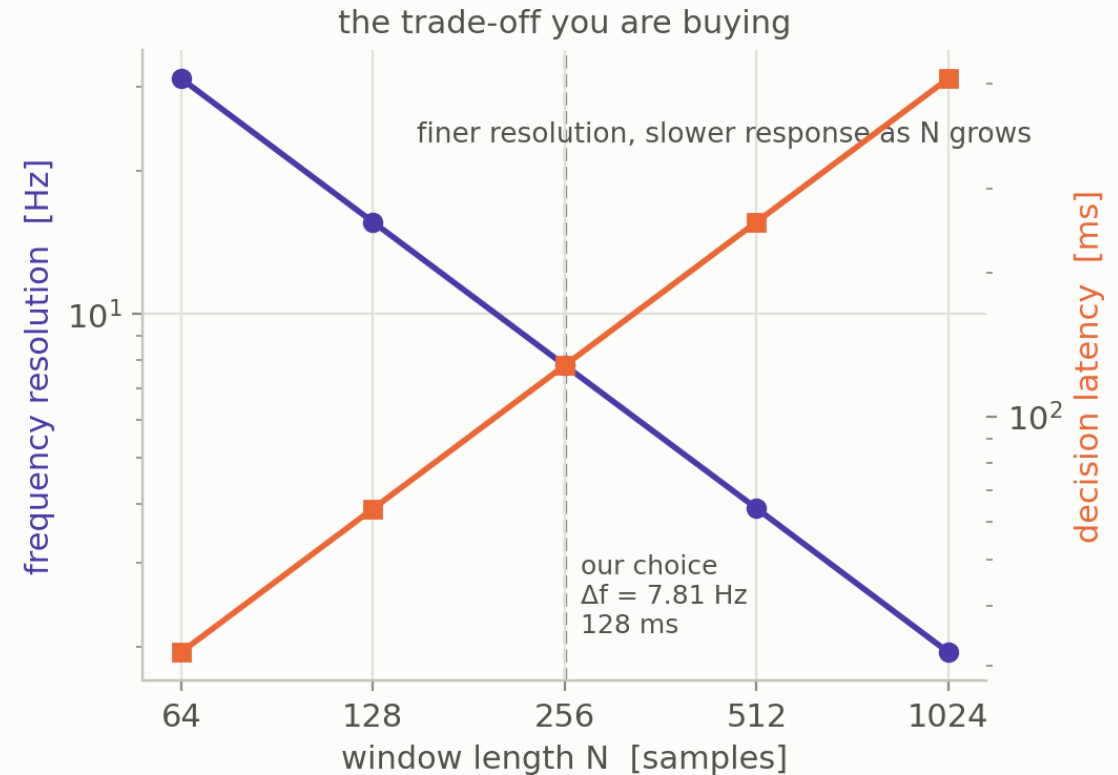
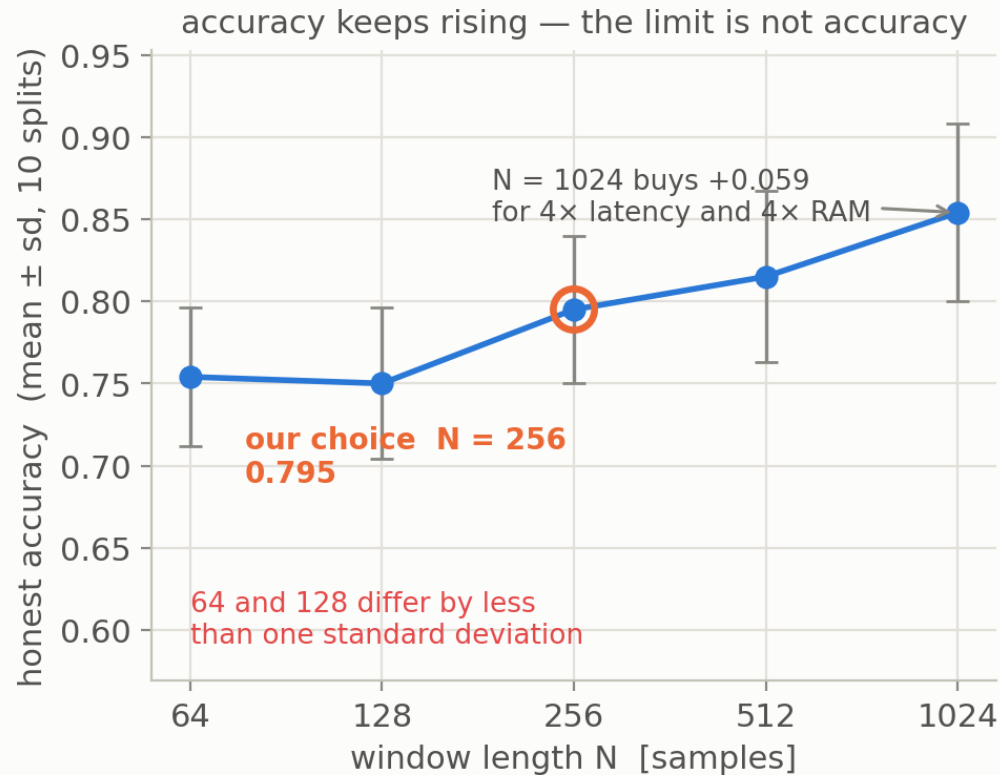
The impulses put their energy where the housing rings, not at the defect frequency itself — so e_bpfo earns no keep.



e_bpfo was designed to catch the defect frequency and carries nothing (0.012 vs 0.014). A feature you invented is a hypothesis, not a fact.

How long should the window be?

- Accuracy keeps climbing with window length, so accuracy alone cannot choose it for you.



N = 256: 7.81 Hz resolution, 128 ms decision latency, 1024 bytes of buffer. N = 1024 buys +0.059 accuracy for 4 $\times$ the latency and 4 $\times$ the RAM.

The split that invents accuracy

- train_test_split shuffles rows. Our rows are not independent: neighbours share samples, and every row of a run carries that run's speed, gain and noise floor.

random split over windows

the 61 windows of one run, in time order

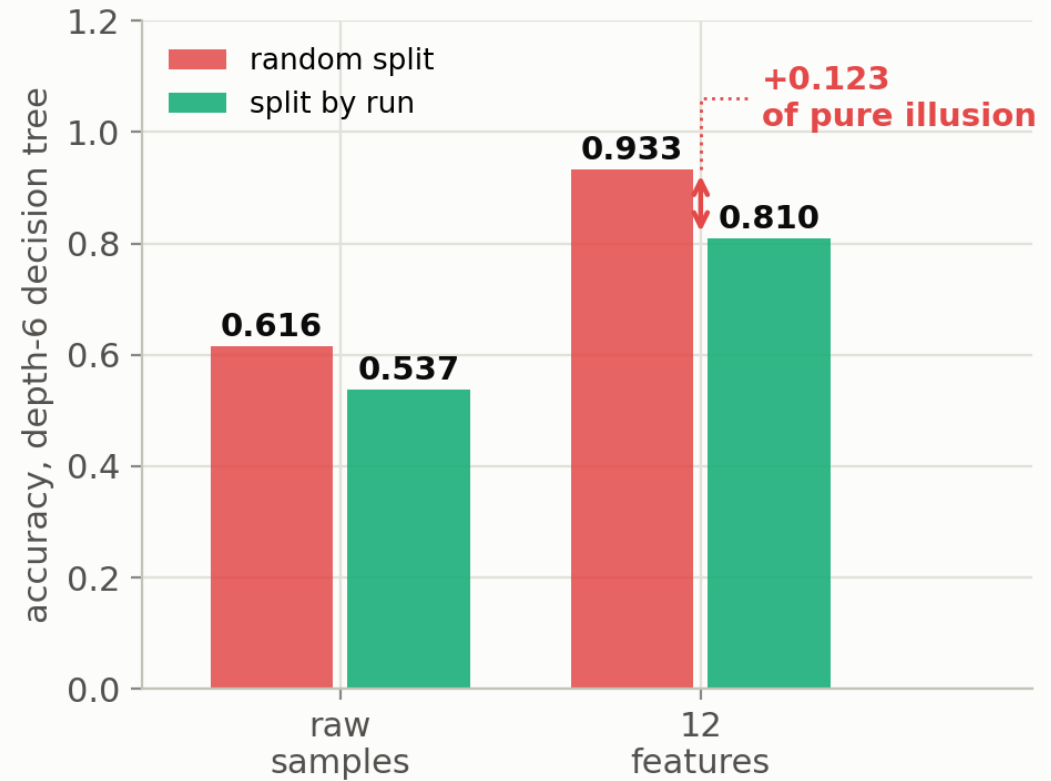


test windows sit between their own training neighbours — the model has already seen half of every test sample

split by run



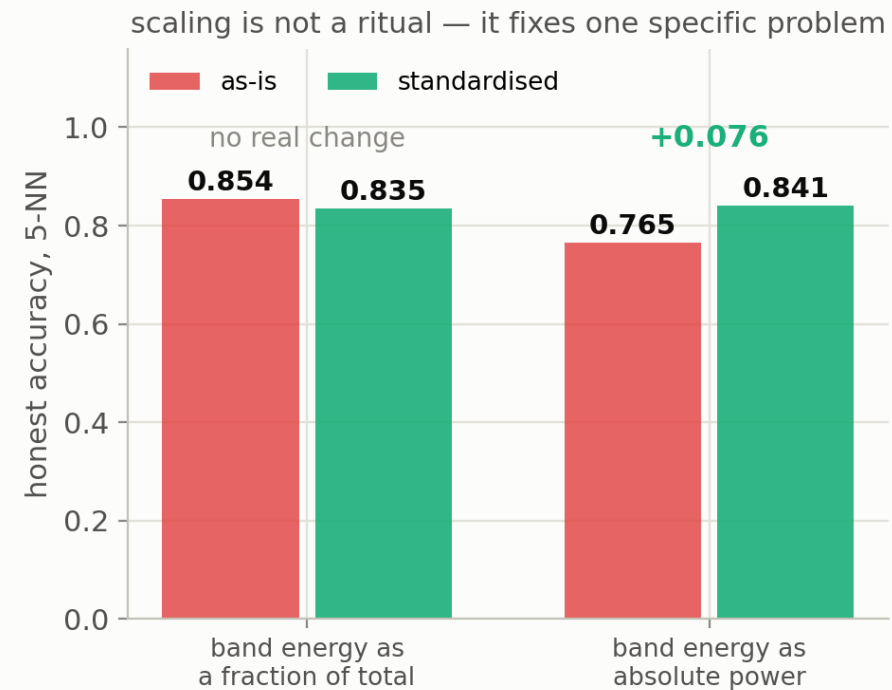
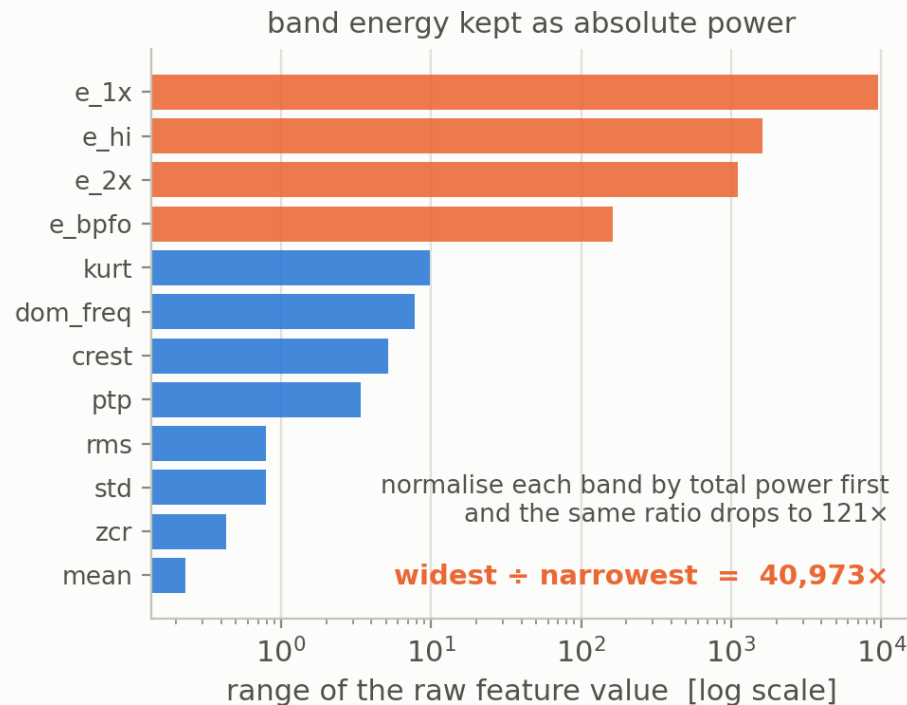
the whole run goes to one side — the only way to answer "will this work on a machine I have never recorded?"



Random split says 0.933. Split by run says 0.810. The difference, +0.123, is entirely fictional — and only the second number answers "will this work on a machine I have never recorded?"

Scaling is not a ritual

- Standardising matters for models that measure distance or regularise coefficients. It does nothing for a decision tree, which only asks “is feature j above t?”.



Standardise when your features have incomparable magnitudes — and fit the scaler on the training runs only, then ship those fixed numbers to the ESP32.

Normalised bands: ranges differ by 121x, scaling changes nothing. Absolute band power: 40,973x, and scaling hands back +0.076. Good feature design removes the problem first.

The same numbers have to come out of the ESP32

- Everything up to the classifier runs on the device in MicroPython. No NumPy, no SciPy — a few dozen lines of plain Python over a 256-sample buffer.

```
# features.py - on the ESP32, MicroPython
def rms_crest_zcr(buf):
    n = len(buf)
    m = sum(buf) / n          # remove DC first
    s = 0.0; pk = 0.0; zc = 0; prev = buf[0] - m
    for v in buf:
        d = v - m
        s += d * d
        if abs(d) > pk: pk = abs(d)
        if (d < 0) != (prev < 0): zc += 1
        prev = d
    r = (s / n) ** 0.5
    return r, pk / r, zc / (n - 1)
```

Full FFT is expensive

A 256-point FFT in pure MicroPython costs 2048 butterflies. We only need four bands — the Goertzel algorithm gets one bin with two state variables.

Identical, not similar

Same window length, same feature order, same scaler constants. If the device computes anything differently, the model sees a world it was never trained on.

Lab 8: stream the stored waveform from flash, window it, print one CSV row per window over the serial port — and check those rows against the notebook to three decimals.

Lab 8 — build the dataset your course will use

On the laptop, in the notebook

Generate the three-state signal set, verify the feature code against NumPy, and export week8_dataset.csv.

On the ESP32, in Wokwi

Upload one 2-second waveform to the device file system, stream it as if it came from the accelerometer, window it, print the seven time-domain features per window — and time the computation against the 128 ms window.

Acceptance test

The device rows and the notebook rows agree to three decimal places — the fourth digit differs because the ESP32 computes in single precision.

Pace for the lab half

Notebook walk-through 60 min · Wokwi build 90 min · device-versus-notebook comparison and write-up 30 min.

```
# device: wave_normal, 4000 samples
win,mean,rms,std,ptp,...
0,-0.0656,0.3007,0.3007,1.5001,...
1,-0.0489,0.2774,0.2774,1.4418,...
2,-0.0313,0.2937,0.2937,1.6930,...
...
# 30 windows
# feature time: ? us per window
# window duration: 128000 us
```

Deliverables

features.py and main.py running in Wokwi · a screenshot of the serial output · week8_dataset.csv · a one-paragraph note on which two features you expect to matter most, and why.

Keep week8_dataset.csv. Week 9 trains on this exact file, and Week 11 deploys the model back onto the device that produced it.

A dataset is a decision, not a download



1. Sample for the fault, not the shaft — everything above $f_s/2$ folds back and lies to you.
2. One window, one row. $N = 256$ costs 128 ms of latency and 1024 bytes; accuracy alone will not choose it.
3. RMS says how much; crest factor and kurtosis say what shape. You need both.
4. A feature you invented is a hypothesis — `e_bpfo` looked essential and carried nothing.
5. Split by run, never at random. Here the random split invents +0.123 of accuracy.
6. Scale when magnitudes are incomparable, fit the scaler on training rows only — and ship those same constants to the ESP32.